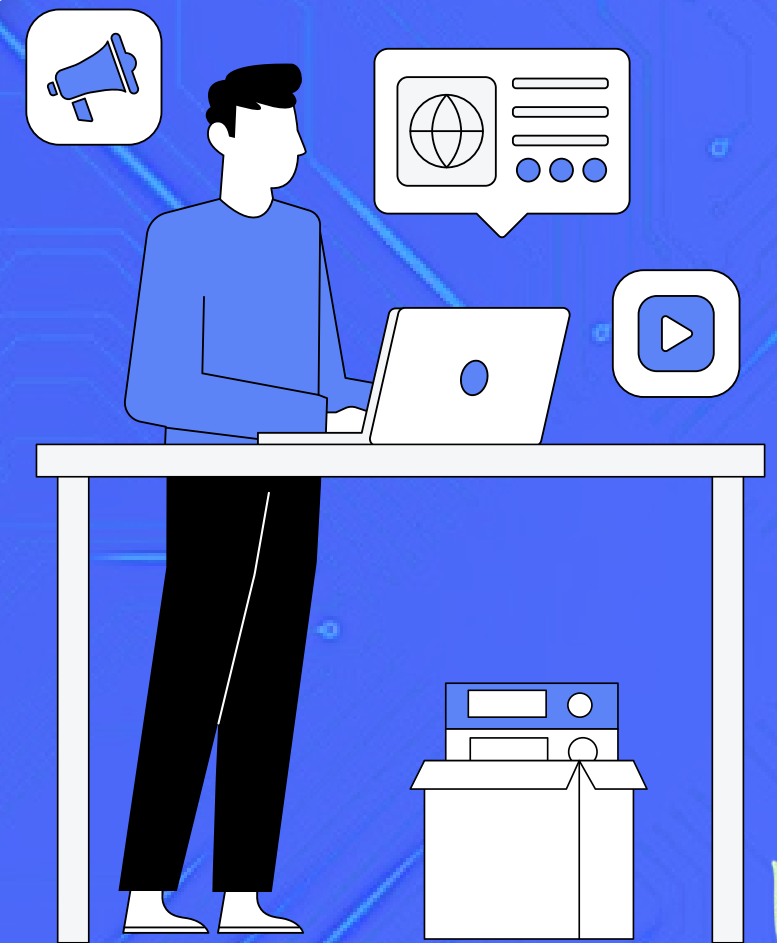


PROJECT ALGORITMA PEMROGRAMAN

OLEH : KELOMPOK B

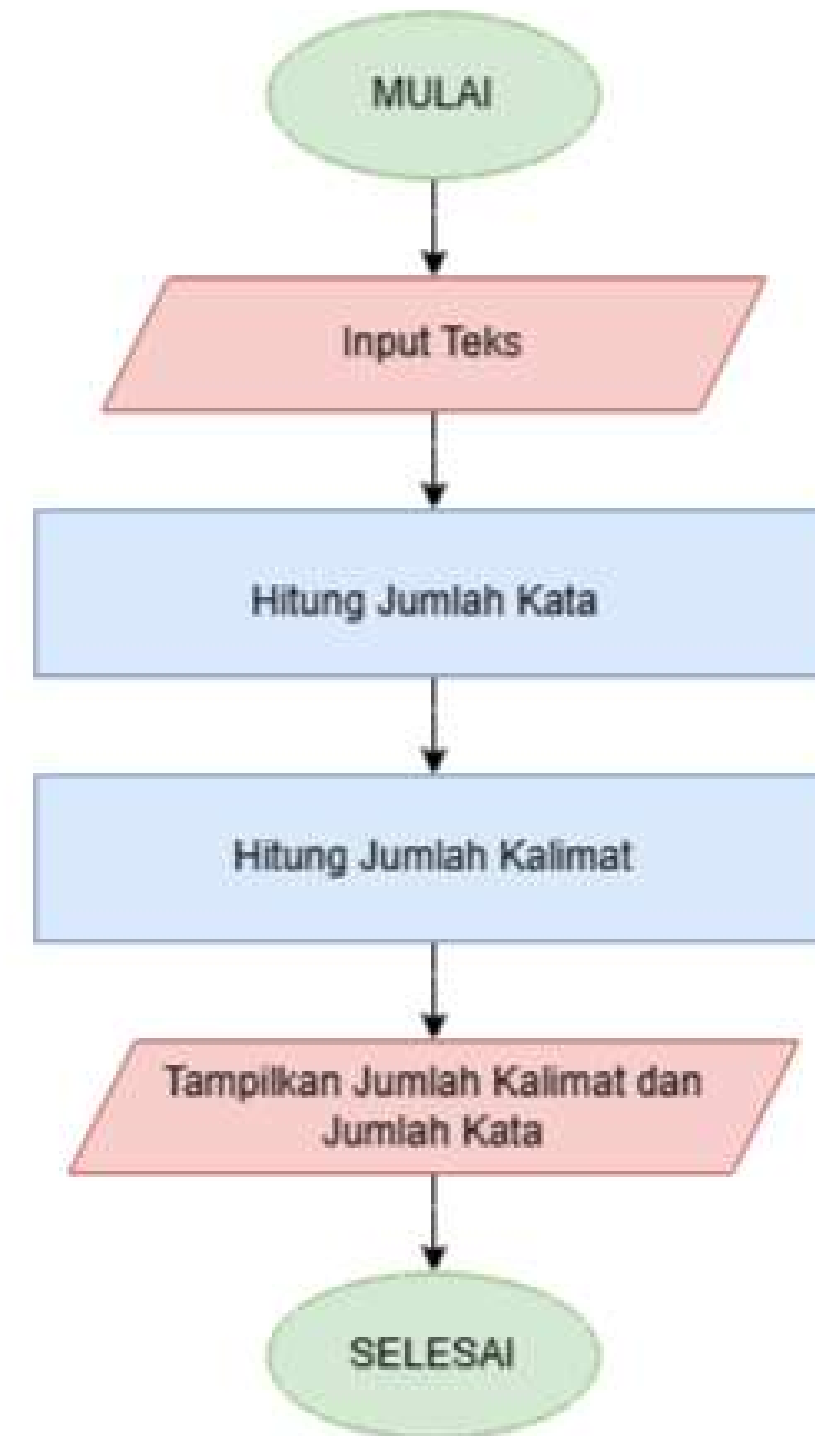
1. NAUFAL LUQYANA ATHOLLAH (K1D025086)
2. NAILA PUTRI ARDIANTI (K1D025089)
3. ZULHILMY MUHAMMAD ASH SHIDQI (K1D025095)
4. MUHAMMAD RAFIF ADRIANTO (K1D025096)
5. INTAN MALIKA SALSABILLA (K1D025105)



DOSEN PENGAMPU : LUTFIAH MAHARANI SINIWI, S.STAT., M.STAT.

PROGRAM 1

Program ini digunakan untuk mencari jumlah kata dan jumlah kalimat dari sebuah teks yang dimasukkan pengguna. Teks yang diinput akan diproses dengan cara memisahkan kata berdasarkan spasi agar jumlah katanya dapat diketahui. Program juga menghitung banyaknya tanda titik (.) karena tanda tersebut dianggap sebagai akhir kalimat. Setelah semua proses selesai, hasil jumlah kata dan kalimat akan ditampilkan. Dengan program ini, pengguna bisa mengetahui isi teks secara lebih mudah dan sederhana.



PROGRAM 1



```
# contoh 1
teks = "Kenaikan harga bensin yang terjadi belakangan ini membuat masyarakat mengeluhkan meningkatnya biaya transportasi dan kebutuhan sehari-hari. Ban
jumlah_kata = len(teks.split())
jumlah_kalimat = teks.count(".")
print("Teks tersebut memuat", jumlah_kalimat,
      "kalimat dan", jumlah_kata, "kata.")
```

Teks tersebut memuat 3 kalimat dan 38 kata.

PROGRAM 2

Program ini dipakai untuk membuat beberapa data string bernama K1, K2, K3, dan K4 sesuai kalimat yang sudah disiapkan. Setiap kalimat dimasukkan ke variabel dengan penulisan tanda kutip yang benar supaya program bisa dijalankan tanpa error. Ada juga penggunaan escape character untuk menampilkan tanda kutip di dalam teks string. Setelah semua variabel selesai dibuat, program akan menampilkan isi variabel tersebut ke layar. Dari program ini, pengguna bisa belajar cara membuat string dan memakai tanda kutip pada bahasa pemrograman Python maupun R



PROGRAM 2

```
20 cat(K3, "\n")
21 cat(K4, "\n")
22
23 #Contoh 3
24 K1 = "Kami tak 'kan melupakan hari itu."
25 K2 = 'Ibu berkata, "Cepat makan siang."'
26 K3 = "\"Apa dampak buruk dari AI?\""
27 K4 = "Kode akses laboratorium adalah LAB/19/2026."
28
29 cat(K1, "\n")
30 cat(K2, "\n")
31 cat(K3, "\n")
32 cat(K4, "\n")
9:14 (Top Level) ↕
```

Console Terminal × Background Jobs ×

R 4.5.1 · ~/

```
"Bagaimana keadaan rupiah saat ini"
> cat(K4, "\n")
Nomor tiket perjalanan adalah TR/97/2026.
>
> #Contoh 3
> K1 = "Kami tak 'kan melupakan hari itu."
> K2 = 'Ibu berkata, "Cepat makan siang."'
> K3 = "\"Apa dampak buruk dari AI?\""
> K4 = "Kode akses laboratorium adalah LAB/19/2026."
>
> cat(K1, "\n")
Kami tak 'kan melupakan hari itu.
> cat(K2, "\n")
Ibu berkata, "Cepat makan siang."
> cat(K3, "\n")
"Apa dampak buruk dari AI?"
> cat(K4, "\n")
Kode akses laboratorium adalah LAB/19/2026.
>
```

PROGRAM 3



Pada persoalan ini, program akan digunakan untuk mencari apakah suatu persamaan kuadrat mempunyai nilai akar-akar real dan menampilkan besar nilai akar-akarnya. Proses yang dilakukan sebelum menghitung besar nilai akar-akar real adalah melakukan pemeriksaan terhadap nilai diskriminan dengan rumus $D = b^2 - 4ac$ untuk mengetahui apakah persamaan memiliki nilai akar-akar real.

```
1 #mencari aka-akar real persamaan kuadrat
2 #mendefinisikan variabel
3 a <- as.numeric(readline(prompt = "masukkan nilai a: "))
4 b <- as.numeric(readline(prompt = "masukkan nilai b: "))
5 c <- as.numeric(readline(prompt = "masukkan nilai c: "))
6
7 #mengidentifikasi persamaan kuadrat
8 if (a == 0) {
9     cat("persamaan bukan persamaan kuadrat\n")
10 } else {
11     Diskriminan <- b^2 - 4*a*c
12 }
13
14 #mencari akar-akar persamaan
15 if (Diskriminan > 0) {
16     x1 <- (-b + sqrt(Diskriminan)) / (2*a)
17     x2 <- (-b - sqrt(Diskriminan)) / (2*a)
18     cat(sprintf("akar-akar real adalah: x1 = %.3f dan x2 = %.3f\n", x1, x2))
19 } else if (Diskriminan == 0) {
20     x1 <- -b / (2*a)
21     x2 <- -b / (2*a)
22     cat(sprintf("persamaan memiliki akar kembar : x1 = %.3f dan x2 = %.3f\n", x1, x2))
23 } else {
24     cat("persamaan hanya mempunyai akar-akar imajiner\n")
25 }
```

PROGRAM 3



The screenshot shows the RStudio environment. The Source pane on the left contains an R script for solving quadratic equations. The Environment pane on the right displays the values of variables defined in the script: a=1, b=-5, c=6, Diskriminan=1, x1=3, and x2=2. The Files, Plots, Packages, Help, Viewer, and Presentation panes are visible at the bottom.

```
> #mencari akar-akar real persamaan kuadrat
> #mendefinisikan variabel
> a <- as.numeric(readline(prompt = "masukkan nilai a: "))
masukkan nilai a: 1
> b <- as.numeric(readline(prompt = "masukkan nilai b: "))
masukkan nilai b: -5
> c <- as.numeric(readline(prompt = "masukkan nilai c: "))
masukkan nilai c: 6
> #mengidentifikasi persamaan kuadrat
> if (a == 0) {
+   cat("persamaan bukan persamaan kuadrat\n")
+ } else {
+   Diskriminan <- b^2 - 4*a*c
+ }
>
> #mencari akar-akar persamaan
> if (Diskriminan > 0) {
+   x1 <- (-b + sqrt(Diskriminan)) / (2*a)
+   x2 <- (-b - sqrt(Diskriminan)) / (2*a)
+   cat(sprintf("akar-akar real adalah: x1 = %.3f dan x2 = %.3f\n", x1, x2))
+ } else if (Diskriminan == 0) {
+   x1 <- -b / (2*a)
+   x2 <- -b / (2*a)
+   cat(sprintf("persamaan memiliki akar kembar : x1 = %.3f dan x2 = %.3f\n", x1, x2))
+ } else {
+   cat("persamaan hanya mempunyai akar-akar imajiner\n")
+ }
akar-akar real adalah: x1 = 3.000 dan x2 = 2.000
> |
```

Values	
a	1
b	-5
c	6
Diskriminan	1
x1	3
x2	2

PROGRAM 4

Program ini dirancang untuk mengekstrak dan menampilkan informasi tanggal lahir seorang Aparatur Sipil Negara (ASN) berdasarkan Nomor Induk Pegawai (NIP) yang dimilikinya. NIP ASN terdiri dari 18 digit angka yang mengandung informasi identitas pegawai secara terstruktur dan terencode di dalamnya.

```
Deklarasi:
nip : string
tahun: string
bulan: integer
tanggal: integer
maks_hari: integer
nama_bulan: string

START

INPUT nip

IF panjang(nip) ≠ 18 ATAU nip bukan semua digit THEN
  OUTPUT "Error: NIP harus terdiri dari tepat 18 digit angka."
  STOP
END IF

tahun ← nip[0:4]
bulan ← integer(nip[4:6])
tanggal ← integer(nip[6:8])

IF bulan < 1 ATAU bulan > 12 THEN
  OUTPUT "Error: Bulan pada NIP tidak valid (harus 01-12)."
  STOP
END IF

IF bulan ∈ {1,3,5,7,8,10,12} THEN
  maks_hari ← 31
ELSE IF bulan ∈ {4,6,9,11} THEN
  maks_hari ← 30
ELSE IF bulan = 2 THEN
  maks_hari ← 29
END IF

IF tanggal < 1 ATAU tanggal > maks_hari THEN
  OUTPUT "Error: Tanggal tidak valid untuk bulan ini (harus 01-maks_hari)."
  STOP
END IF
```

```
IF bulan = 1 THEN nama_bulan ← "Januari"
ELSE IF bulan = 2 THEN nama_bulan ← "Februari"
ELSE IF bulan = 3 THEN nama_bulan ← "Maret"
ELSE IF bulan = 4 THEN nama_bulan ← "April"
ELSE IF bulan = 5 THEN nama_bulan ← "Mei"
ELSE IF bulan = 6 THEN nama_bulan ← "Juni"
ELSE IF bulan = 7 THEN nama_bulan ← "Juli"
ELSE IF bulan = 8 THEN nama_bulan ← "Agustus"
ELSE IF bulan = 9 THEN nama_bulan ← "September"
ELSE IF bulan = 10 THEN nama_bulan ← "Oktober"
ELSE IF bulan = 11 THEN nama_bulan ← "November"
ELSE IF bulan = 12 THEN nama_bulan ← "Desember"
END IF
OUTPUT "Tanggal lahir: " + tanggal + " " + nama_bulan + " " + tahun
END
```

PROGRAM 4

```
1 #program 4
2 nip <- readline("Masukkan NIP ASN: ")
3
4 if (nchar(nip) != 18 || grepl("[^0-9]", nip)) {
5   cat("Error: NIP harus terdiri dari tepat 18 digit angka.\n")
6 } else {
7   tahun  <- substr(nip, 1, 4)
8   bulan  <- as.integer(substr(nip, 5, 6))
9   tanggal <- as.integer(substr(nip, 7, 8))
10
11   if (bulan < 1 || bulan > 12) {
12     cat("Error: Bulan pada NIP tidak valid (harus 01-12).\n")
13   } else {
14     if (bulan %in% c(1, 3, 5, 7, 8, 10, 12)) {
15       maks_hari <- 31
16     } else if (bulan %in% c(4, 6, 9, 11)) {
17       maks_hari <- 30
18     } else if (bulan == 2) {
19       maks_hari <- 29
20     }
21
22     if (tanggal < 1 || tanggal > maks_hari) {
23       cat(paste0("Error: Tanggal pada NIP tidak valid untuk bulan ini (harus 01-",
24                 sprintf("%02d", maks_hari), ").\n"))
25     } else {
26       if (bulan == 1) {
27         nama_bulan <- "Januari"
28       } else if (bulan == 2) {
29         nama_bulan <- "Februari"
```

19:22 (Top Level) ⚡

Console Terminal × Render × Background Jobs ×

R 4.5.1 · ~/

```
> source("C:/Users/Pongo/Downloads/project_alpro 4/project ALPRO soal 4 R.R")
Masukkan NIP ASN: 123456789
Error: NIP harus terdiri dari tepat 18 digit angka.
> source("C:/Users/Pongo/Downloads/project_alpro 4/project ALPRO soal 4 R.R")
Masukkan NIP ASN: 198778120934874560
Error: Bulan pada NIP tidak valid (harus 01-12).
> source("C:/Users/Pongo/Downloads/project_alpro 4/project ALPRO soal 4 R.R")
Masukkan NIP ASN: 204508170012436587
Tanggal lahir: 17 Agustus 2045
> |
```



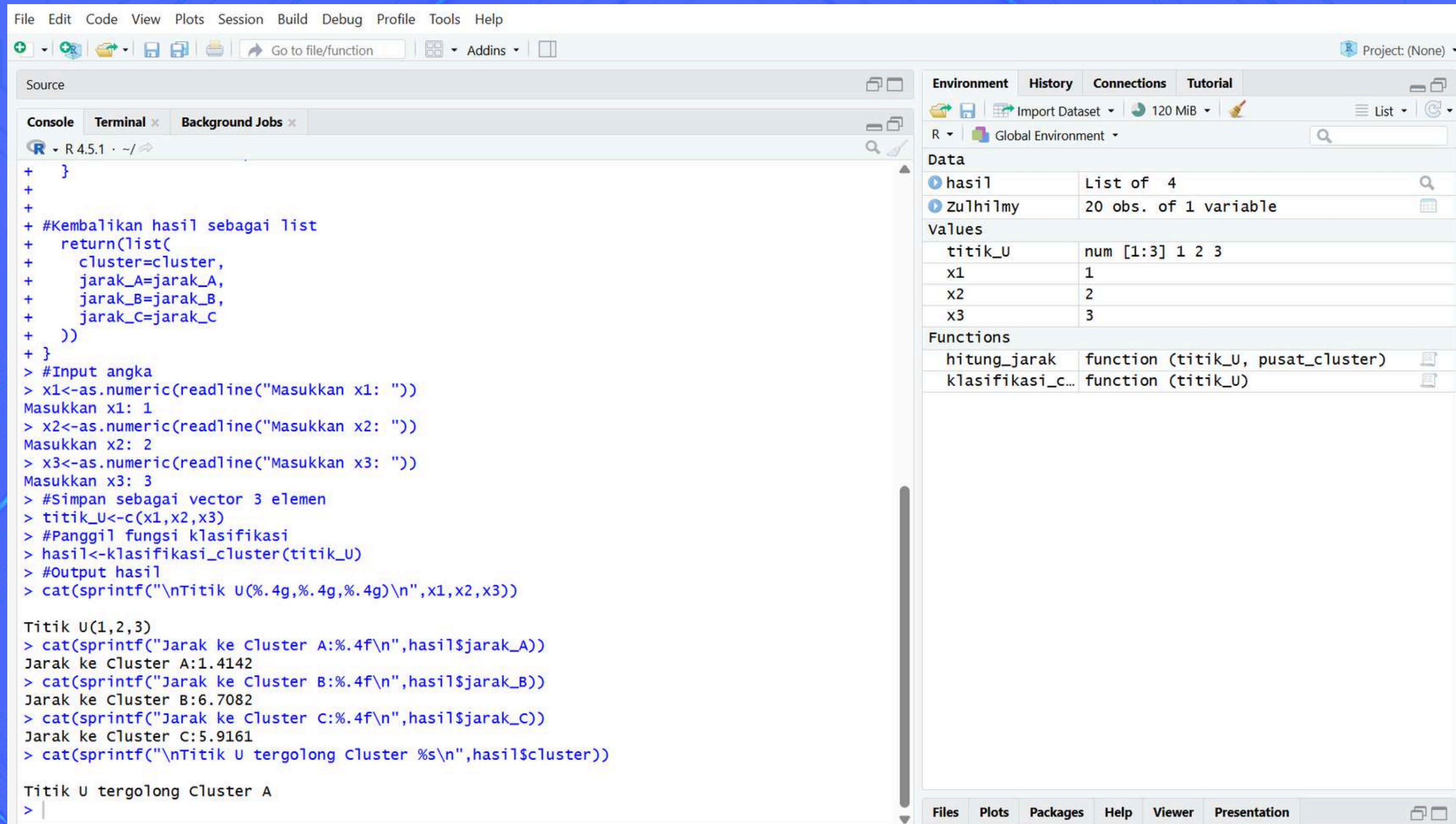
PROGRAM 5

PROGRAM INI MENERIMA SEBUAH TITIK SEMBARANG DALAM RUANG 3 DIMENSI BERBENTUK TUPLE (X1,X2,X3). PROGRAM KEMUDIAN MENGHITUNG JARAK EUCLID TERSEBUT TERHADAP TIGA PUSAT CLUSTER A, B, DAN C YANG TELAH DITENTUKAN. CLUSTER YANG MEMILIKI JARAK PALING KECIL MENJADI HASIL KLASIFIKASI TITIK TERSEBUT DAN JIKA TERDAPAT JARAK YANG SAMA, MAKA TIDAK TERMASUK KE DALAM SALAH SATU CLUSTER.

```
1 #Program Klasifikasi Cluster 3 Dimensi
2 #FUNGSI: menghitung jarak Euclidean antara dua titik 3 Dimensi
3 hitung_jarak <- function(titik_U, pusat_cluster) {
4   x1<-titik_U[1];y1<-pusat_cluster[1]
5   x2<-titik_U[2];y2<-pusat_cluster[2]
6   x3<-titik_U[3];y3<-pusat_cluster[3]
7
8   jarak<-sqrt(
9     (x1-y1)^2+
10    (x2-y2)^2+
11    (x3-y3)^2
12  )
13  return(jarak)
14 }
15
16 #Fungsi menentukan cluster dari titik U
17 klasifikasi_cluster<-function(titik_U){
18   #Konstanta pusat masing-masing cluster
19   pusat_A<-c(2,1,3)
20   pusat_B<-c(1,-4,6)
21   pusat_C<-c(-2,3,-2)
22
23   #Hitung jarak ke tiap pusat cluster
24   jarak_A<-hitung_jarak(titik_U,pusat_A)
25   jarak_B<-hitung_jarak(titik_U,pusat_B)
26   jarak_C<-hitung_jarak(titik_U,pusat_C)
27
28   #Tentukan cluster terdekat
29   if(jarak_A<jarak_B&&jarak_A<jarak_C){
30     cluster<-"A"
31   }else if(jarak_B<jarak_A&&jarak_B<jarak_C){
32     cluster<-"B"
33   }else if(jarak_C < jarak_A && jarak_C < jarak_B){
34     cluster<-"C"
```

```
34   cluster<-"C"
35 }else{
36   cluster<-"tidak dapat ditentukan"
37 }
38
39
40 #Kembalikan hasil sebagai list
41 return(list(
42   cluster=cluster,
43   jarak_A=jarak_A,
44   jarak_B=jarak_B,
45   jarak_C=jarak_C
46 ))
47 }
48
49 #Input angka
50 x1<-as.numeric(readline("Masukkan x1: "))
51 x2<-as.numeric(readline("Masukkan x2: "))
52 x3<-as.numeric(readline("Masukkan x3: "))
53
54 #Simpan sebagai vector 3 elemen
55 titik_U<-c(x1,x2,x3)
56
57 #Panggil fungsi klasifikasi
58 hasil<-klasifikasi_cluster(titik_U)
59
60 #Output hasil
61 cat(sprintf("\nTitik U(%.4g,%.4g,%.4g)\n",x1,x2,x3))
62 cat(sprintf("Jarak ke Cluster A:%.4f\n",hasil$jarak_A))
63 cat(sprintf("Jarak ke Cluster B:%.4f\n",hasil$jarak_B))
64 cat(sprintf("Jarak ke Cluster C:%.4f\n",hasil$jarak_C))
65 cat(sprintf("\nTitik U tergolong Cluster %s\n",hasil$cluster))
```

PROGRAM 5



The screenshot displays the RStudio interface with the following components:

- Source Editor:** Contains R code for a clustering function and its execution. The code defines a function `klasifikasi_cluster` that takes `titik_U` and `pusat_cluster` as arguments. It calculates distances and assigns points to clusters based on the minimum distance.
- Console:** Shows the execution of the code, including user input for `x1`, `x2`, and `x3`, and the resulting output for the clustering function.
- Environment:** Lists the objects in the global environment, including `hasil` (a list) and `Zulhilmy` (a variable).
- Values:** Displays the values of the objects, showing `titik_U` as a numeric vector `num [1:3] 1 2 3`.
- Functions:** Lists the functions defined in the environment, including `hitung_jarak` and `klasifikasi_c...`.

```
+ }  
+  
+ #Kembalikan hasil sebagai list  
+ return(list(  
+   cluster=cluster,  
+   jarak_A=jarak_A,  
+   jarak_B=jarak_B,  
+   jarak_C=jarak_C  
+ ))  
+ }  
> #Input angka  
> x1<-as.numeric(readline("Masukkan x1: "))  
Masukkan x1: 1  
> x2<-as.numeric(readline("Masukkan x2: "))  
Masukkan x2: 2  
> x3<-as.numeric(readline("Masukkan x3: "))  
Masukkan x3: 3  
> #Simpan sebagai vector 3 elemen  
> titik_U<-c(x1,x2,x3)  
> #Panggil fungsi klasifikasi  
> hasil<-klasifikasi_cluster(titik_U)  
> #Output hasil  
> cat(sprintf("\nTitik U(%.4g,%.4g,%.4g)\n",x1,x2,x3))  
  
Titik U(1,2,3)  
> cat(sprintf("Jarak ke Cluster A:%.4f\n",hasil$jarak_A))  
Jarak ke Cluster A:1.4142  
> cat(sprintf("Jarak ke Cluster B:%.4f\n",hasil$jarak_B))  
Jarak ke Cluster B:6.7082  
> cat(sprintf("Jarak ke Cluster C:%.4f\n",hasil$jarak_C))  
Jarak ke Cluster C:5.9161  
> cat(sprintf("\nTitik U tergolong Cluster %s\n",hasil$cluster))  
  
Titik U tergolong Cluster A  
> |
```

Environment | History | Connections | Tutorial

R | Global Environment

Data

Object	Value
hasil	List of 4
Zulhilmy	20 obs. of 1 variable

Values

Object	Value
titik_U	num [1:3] 1 2 3
x1	1
x2	2
x3	3

Functions

Object	Value
hitung_jarak	function (titik_U, pusat_cluster)
klasifikasi_c...	function (titik_U)

Files | Plots | Packages | Help | Viewer | Presentation

PROGRAM 6

Pada program 6, program ini digunakan untuk menghitung interval konfidensi proporsi berdasarkan data sampel yang diberikan. Data yang diperlukan untuk menjalankan program ini yaitu nilai proporsi sampel (p_{hat}), ukuran sampel (n), dan tingkat signifikansi (α). Nilai-nilai tersebut akan diolah dengan menentukan nilai z sesuai tingkat signifikansi yang dipilih, kemudian menghitung margin of error menggunakan rumus interval konfidensi proporsi. Selanjutnya, program akan menghitung batas bawah dan batas atas interval konfidensi berdasarkan nilai margin of error yang diperoleh. Dengan adanya program ini, pengguna dapat lebih mudah menentukan rentang estimasi proporsi populasi berdasarkan data sampel yang dimiliki.

START

DEKLARASI:

```
p_hat : Real
n : Integer
alpha : Real
z : Real
margin_error : Real
batas_bawah : Real
batas_atas : Real
hasil : Record {BB, BA}
```

DESKRIPSI:

```
Input p_hat
Input n
Input alpha
```

```
IF p_hat < 0 OR p_hat > 1 THEN
  Tampilkan "Error: Proporsi sampel harus berada pada rentang 0 sampai 1"
ELSE IF n <= 0 THEN
  Tampilkan "Error: Ukuran sampel harus lebih dari 0"
ELSE IF alpha ≠ 0.05 AND alpha ≠ 0.10 THEN
  Tampilkan "Error: Alpha harus bernilai 0.05 atau 0.10"
ELSE
  IF alpha = 0.05 THEN
    z = 1.96
  ELSE IF alpha = 0.10 THEN
    z = 1.645
  ENDIF
```

```
margin_error ←  $z \times \sqrt{(p_{\text{hat}} \times (1 - p_{\text{hat}})) / n}$ 
```

```
batas_bawah ← p_hat - margin_error
batas_atas ← p_hat + margin_error
```

```
hasil.BB ← batas_bawah
hasil.BA ← batas_atas
```

```
Tampilkan "Batas bawah = ", hasil.BB
Tampilkan "Batas atas = ", hasil.BA
Tampilkan "Interval konfidensi = (", hasil.BB, ", ", hasil.BA, ")"
ENDIF
```

END

PROGRAM 6

```
#KONDISI 1
import math

# Fungsi
def hitung_interval(p_hat, n, alpha):

    if alpha == 0.05:
        z = 1.96
    elif alpha == 0.10:
        z = 1.645

    margin_error = z * math.sqrt((p_hat * (1 - p_hat)) / n)

    batas_bawah = p_hat - margin_error
    batas_atas = p_hat + margin_error

    return {
        "BB": batas_bawah,
        "BA": batas_atas
    }

# Input
p_hat = 0.567
n = 10
alpha = 0.05

# Validasi
if p_hat < 0 or p_hat > 1:
    print("Error: Proporsi sampel harus berada pada rentang 0 sampai 1")

elif n <= 0:
    print("Error: Ukuran sampel harus lebih dari 0")

elif alpha != 0.05 and alpha != 0.10:
    print("Error: Alpha harus bernilai 0.05 atau 0.10")

else:
    hasil = hitung_interval(p_hat, n, alpha)

    # Menampilkan Hasil
    print("Batas Bawah =", hasil["BB"])
    print("Batas Atas =", hasil["BA"])
    print("Interval konfidensi = (", round(hasil["BB"], 4), ",", round(hasil["BA"], 4), ")")

Batas Bawah = 0.25989170353114843
Batas Atas = 0.8741082964688515
Interval konfidensi = ( 0.2599 , 0.8741 )
```